PRACTICAL NO : 4

Name: Payal Ganesh Ghanwat Roll No:222 Batch: S1 Branch:SE(AIML)

Problem Statement: Binary Search Tree -- CO1, CO2, CO3

Implement binary search tree and perform following operations:

a. Insert (Handle insertion of duplicate entry)

b. Delete

c. Search

d. Display tree (Traversal)

e. Display - Depth of tree

f. Display - Mirror image

g. Create a copy

h. Display all parent nodes with their child nodes

i. Display leaf nodes

j. Display tree level wise

CODE

```
# Node Class
class Node:
def __init__(self, data):
self.data = data
self.left = None
self.right = None

# BST Class
class BST:

def __init__(self):
self.root = None

# Insert
def insert(self, node, data):
if node is None:
```

```
return Node(data)

if data < node.data:
node.left = self.insert(node.left, data)

elif data > node.data:
node.right = self.insert(node.right, data)

else:
print("Duplicate Entry Not Allowed")

return node

# Search
def search(self, node, key):
if node is None:

return False

if node.data == key:
return True

if key < node.data:
return self.search(node.left, key)

return self.search(node.right, key)

# Find Minimum
def minimum(self, node):
while node.left:
node = node.left
```

```
return node

# Delete
def delete(self, node, key):

if node is None:
return node

if key < node.data:
node.left = self.delete(node.left, key)

elif key > node.data:

node.right = self.delete(node.right, key)

else:

if node.left is None:
return node.right

elif node.right is None:
return node.left

temp = self.minimum(node.right)

node.data = temp.data

node.right = self.delete(node.right, temp.data)

return node
```

```
# Inorder
def inorder(self, node):
if node:
self.inorder(node.left)
print(node.data, end=" ")
self.inorder(node.right)

# Preorder

def preorder(self, node):
if node:
print(node.data, end=" ")
self.preorder(node.left)
self.preorder(node.right)

# Postorder
def postorder(self, node):
if node:
self.postorder(node.left)
self.postorder(node.right)
print(node.data, end=" ")

# Depth
def depth(self, node):
if node is None:
return 0

return max(self.depth(node.left),
self.depth(node.right)) + 1

# Mirror
```

```
def mirror(self, node):
if node:
node.left, node.right = node.right, node.left
self.mirror(node.left)

self.mirror(node.right)

# Parent with Children
def parentChild(self, node):

if node:

print("Parent:", node.data, end=" -> ")

if node.left:
print("Left:", node.left.data, end=" ")

if node.right:
print("Right:", node.right.data, end=" ")

print()

self.parentChild(node.left)
self.parentChild(node.right)

# Leaf Nodes
def leafNodes(self, node):

if node:

if node.left is None and node.right is None:
```

```
print(node.data, end=" ")

self.leafNodes(node.left)
self.leafNodes(node.right)

# Level Order Traversal
def levelOrder(self):

if self.root is None:
return

q = deque()

q.append(self.root)

while q:

node = q.popleft()

print(node.data, end=" ")

if node.left:
q.append(node.left)

if node.right:
q.append(node.right)

# Main Program
bst = BST()
```

```
while True:

print("\n------ Binary Search Tree ------")
print("1. Insert")
print("2. Delete")
print("3. Search")
print("4. Inorder Traversal")
print("5. Preorder Traversal")
print("6. Postorder Traversal")
print("7. Display Depth")
print("8. Mirror Image")
print("9. Copy Tree")
print("10. Parent with Child")
print("11. Leaf Nodes")
print("12. Level Order")
print("13. Exit")

choice = int(input("Enter Choice: "))

if choice == 1:
x = int(input("Enter Value: "))

bst.root = bst.insert(bst.root, x)

elif choice == 2:
x = int(input("Enter Value to Delete: "))
bst.root = bst.delete(bst.root, x)

elif choice == 3:
x = int(input("Enter Value to Search: "))
if bst.search(bst.root, x):
```

```
print("Element Found")
else:
print("Element Not Found")

elif choice == 4:
print("Inorder:")
bst.inorder(bst.root)
print()

elif choice == 5:
print("Preorder:")
bst.preorder(bst.root)
print()

elif choice == 6:
print("Postorder:")
bst.postorder(bst.root)

print()

elif choice == 7:
print("Depth =", bst.depth(bst.root))

elif choice == 8:
bst.mirror(bst.root)
print("Mirror Image Created")
print("Inorder after Mirror:")
bst.inorder(bst.root)
print()

elif choice == 9:
```

```
newTree = copy.deepcopy(bst)
print("Tree Copied Successfully")

elif choice == 10:
print("Parent -> Child")
bst.parentChild(bst.root)

elif choice == 11:
print("Leaf Nodes:")
bst.leafNodes(bst.root)
print()

elif choice == 12:

print("Level Order Traversal:")
bst.levelOrder()
print()

elif choice == 13:
print("Program Ended")
break

else:
print("Invalid Choice")
```

Output:

```
------ Binary Search Tree ------
1. Insert
2. Delete
3. Search
4. Inorder Traversal
```

5. Preorder Traversal

6. Postorder Traversal

7. Display Depth

8. Mirror Image

9. Copy Tree

10. Parent with Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice: 1

Enter Value: 50

Enter Choice: 1

Enter Value: 30

Enter Choice: 1

Enter Value: 70

Enter Choice: 1

Enter Value: 20

Enter Choice: 4

Inorder:

20 30 50 70

Enter Choice: 3

Enter Value to Search: 30

Element Found

Enter Choice: 7

Depth = 3

Enter Choice: 11

Leaf Nodes:

20 70

Enter Choice: 12

Level Order Traversal:

50 30 70 20

Enter Choice: 13

Program Ended